

Keeping Architecture on Cadence: Skill-Orchestrated ArchiMate Modelling in the System of Record

JG Systems Consulting Ltd

Abstract—Delivery teams ship weekly. Architecture views lag by sprints. The lag is not a shortage of ArchiMate or of Archi: the modelling method lives in the architect’s head and the tool is a canvas. Skill-Orchestrated ArchiMate Modelling (SOAM) is a governed pipeline: an orchestrator elicits intent and a view plan, the architect approves, then specialists write only into Archi through an MCP bridge. The ArchiMate language stays in MCP resources rather than in the agent. Two author-run frozen briefs demonstrate the pipeline: a mill CRM programme that must not absorb the MES (Hatherley Plate), and a training-range planning tool that must not absorb the GCS or the airborne computer (Moorfield Range). The defence is demonstration, not timing. The claim is method, not a speedup percentage.

Index Terms—enterprise architecture, ArchiMate, Archi, modelling tools, large language models, Model Context Protocol, human in the loop

1 INTRODUCTION

Enterprise architecture is supposed to be the system of record for change. In an engineering organisation it often is not. In practice ArchiMate work is an unrecorded process. The architect runs a workshop, chooses a viewpoint without recording the choice, and places boxes in Archi for days. Review finds the wrong cut. The rationale ends up in slides the model never holds.

ISO/IEC/IEEE 42010 already said stakeholders get viewpoints [1]. ArchiMate defined them [2]. Archi holds a real model [3]. The bottleneck is not the language; the method lives in the modeller’s head and the tool is a canvas. Delivery teams ship weekly. Architecture views lag by sprints. That lag is a throughput problem: architecture cannot keep cadence with engineering because the method is uninstrumented. Throughput here is relevance, not a measured result. Skill-Orchestrated ArchiMate Modelling (SOAM) is the artefact offered against that problem: architect-governed, system-of-record native, and MCP-instrumented.

SOAM is a human-governed method: orchestrator-dispatched specialists for elicitation, viewpoint selection, layer modelling, traceability, quality assurance, layout, and documentation, writing only into Archi through an MCP bridge, with the ArchiMate language kept in MCP resources rather than in the agent [4]–[6]. Coding-agent hosts (ZCode, Claude Code) are runtime, not the artefact.

We do not claim that ArchiMate is incomplete, that TO-GAF is the problem, or that architects are slow. Given ArchiMate and Archi, dominant practice does not instrument the method, so architecture cannot keep cadence with engineering. Language-model modelling papers mostly emit a parallel artefact [7], [8]. EA suites are still mostly undriveable

by an agent without forking the language into a prompt. SOAM attacks both gaps at once.

This paper is for staff engineers and delivery architects who know ArchiMate and have watched views fall behind the backlog. It is not a language tutorial. It accounts for why the method never made it into the tool, and one way to put it there without surrendering the gate.

Section 2 describes traditional ArchiMate work as an engineering organisation runs it. Section 3 places SOAM against viewpoints, model quality, language-model modelling, and method engineering. Section 4 states the method rules, pipeline, specialist contract, and MCP instrument. Section 5 maps waste classes to pipeline stages and walks two frozen briefs plus a refusal control. Section 6 covers adoption and threats to validity. Section 7 concludes.

2 TRADITIONAL ARCHIMATE WORK IN AN ENGINEERING ORGANISATION

A typical cycle in a shop that already owns Archi starts when a programme sponsor asks for “the architecture” of a change: a second CRM, a landing zone, a quote-to-cash cut. Someone books a workshop. Notes go into a document. An architect later reconstructs intent in Archi from those notes, last quarter’s views, and memory of which relationships the language allows. Viewpoint choice is rarely written down. The first view that appears on a shared screen is often a kitchen sink: business actors, applications, and a node or two, because that is what fitted last time.

Review is late. Stakeholders see the drawing after the boxes exist. The wrong audience is found when the COO cannot find a capability cut and IT cannot find the integration. Rework is placing boxes again, not rerunning a method. Cross-layer traces (capability to process to application to node to work package) are the first labour dropped under a full programme week. Rationale, if it survives, lives

in a slide that will not be opened when the model is next copied.

That loop is an uninstrumented method on a canvas tool. ArchiMate and 42010 already told the architect what a viewpoint is [1], [2]. They did not put elicitation, viewpoint selection, correspondence, QA, layout, and documentation on a critical path the tool can run. Engineering organisations feel this as lag: delivery ships weekly; architecture views arrive a sprint later, or not at all, and the team proceeds on tribal knowledge.

Language-model assistants entered this gap as diagram generators. The failure mode is the opposite of the canvas problem. Generation is fast; the artefact is not the model. The architect is asked to approve a picture. Illegal relationships are a prompt accident. The Archi file operations and the next programme will open is unchanged [7], [8]. The line has reached ArchiMate itself [9], and metamodel handling remains the weak point of prompt-carried language [10].

SOAM is an approach that neither adds drawing time nor produces a second artefact. It writes into Archi after a human gate, with the language left on the MCP.

3 RELATED WORK

3.1 Viewpoints and EA practice

ISO/IEC/IEEE 42010 frames architecture description as stakeholders, concerns, viewpoints, and correspondence [1]. ArchiMate supplies a layered language and a catalogue of viewpoints for those concerns [2]. Lankhorst treats enterprise architecture as a working practice, not a drawing hobby [11]. Greefhorst and Proper put architecture principles at the centre of that practice [12]. Proper and Lankhorst argue that EA is essential sensemaking for the organisation [13]. Jonkers et al. call the architecture both a management tool and a blueprint [14]. TOGAF's Architecture Development Method, taken lightly, is a cycle of architecture work rather than a drawing task [15]. Archi is the open-source toolkit where many practitioners place those views [3].

The language and the tool exist; the method still lives in the modeller's head. Viewpoint choice stays tacit; correspondence stays optional. A practitioner copies last week's view, fills an overloaded diagram, and finds the wrong audience in review. Nothing in 42010 or the ArchiMate specification executes that choice inside the tool.

3.2 Model quality and correspondence

Moody's physics of notations shows that visual languages fail as communication when graphic economy and cognitive load are ignored [16]. Quartel et al. showed that goal-oriented motivation can be modelled in the EA stack rather than left in a separate requirements pile [17]. Hacks et al. named enterprise-architecture debt as the residue of postponed correspondence and quality work [18]. In ArchiMate practice the allowed-relationship table, completeness of traces, and layout of a view are the daily quality problems.

Checks and style guides run after the modelling: draw, check, then paste documentation into a slide. SOAM puts traceability, quality assurance, and layout on the critical path after layer modelling and before the run is treated as a draft.

3.3 Language-model assisted modelling

Cámara et al. assessed ChatGPT on UML modelling tasks and reported mixed reliability outside a controlled prompt [7]. Fill, Fettke, and Köpke reported conceptual-modelling experiments in which models emit plausible diagrams that are not the organisation's model [8]. Surveys cover the line: Di Rocco et al. map LLM use across model-driven engineering [19], and a BISE editorial by Fill, Horkoff, Fettke, and Köpke names generative conceptual modelling a research direction [20]. Nast et al. generate ArchiMate models from natural language [9]; Chen and Zhao dispatch multiple agents on enterprise-architecture design [21]. A growing set of papers generate PlantUML, JSON, or pictures from text. Some wrap a drawing tool. The Model Context Protocol is the open tool-and-resource interface those agents can speak [6], [22]. It is instrumentation, not a modelling method.

Those papers emit a parallel artefact. They copy or approximate the metamodel in the prompt. The system of record is unchanged. SOAM writes into Archi. Language source of truth stays in MCP resources. Architect approval of a view plan is a method rule, not a UI confirm on a generated picture.

3.4 Design science and method engineering

Hevner et al. locate design-science contributions in artefacts evaluated against a problem [23]. Peffers et al. license demonstration plus informed argument as evaluation for a first method paper [24]. Brinkkemper defined method engineering as the engineering of methods and tools [25]. Ralyté, Deneckère, and Rolland treat situational method engineering as composition of fragments under a situation [26].

Design science licenses demonstration plus informed argument. Situational method engineering treats fragments and composition. Neither supplies an architect-governed, system-of-record-native modelling pipeline. SOAM uses specialists as method fragments and the orchestrator as composition. This paper is not an SME treatise. The evaluation demonstrates that those fragments can be dispatched against named waste classes, with the architect still on the gate.

Table 1 is the comparison an engineer needs. Traditional ArchiMate practice keeps the system of record and loses the method. Language-model diagram generation keeps a method-shaped prompt and loses the system of record. SOAM keeps both.

4 SKILL-ORCHESTRATED ARCHIMATE MODELLING

SOAM is a method: an orchestrator, twelve specialists, and a consume-only MCP bridge into Archi [4], [5]. The language is not in the skills.

4.1 Five rules

- 1) **The user governs.** Input is plain-language intent. Specialists do not mutate until the architect approves a view plan naming views, stakeholders,

TABLE 1
Where the model lives, who governs, and what the language source of truth is.

	Traditional Archi practice	LLM diagram generation	SOAM
System of record	Archi, if the architect finishes	Parallel PlantUML, JSON, or picture	Archi, written through MCP
Viewpoint choice	Tacit; last week's view	Prompt or overloaded diagram	Viewpoint-select, then approval
When written	After workshops, over days	Immediately, outside Archi	After view-plan approval
Language SoT	Spec on the shelf	Copied into the prompt	MCP resources
Correspondence	Optional labour	Rarely a stage	Traceability on the critical path
Documentation	Slide deck	Caption on the picture	Fields on the elements
Architect authority	High, slow	Low, after the fact	High, as a gate before mutate

concerns, and layer specialists. Scope refusals are method behaviour. A refused redesign is not a failed generation. The architect can reject and return to elicitation with no half-written model.

- 2) **The orchestrator composes; specialists execute.** Elicitation and viewpoint selection run first; only named layers are dispatched; the user does not invoke layer skills. That rule stops modelling every layer: a capability cut does not silently grow a technology view, and a landing-zone move does not invent a new operating model to justify the diagram.
- 3) **Archi is the only canvas.** Elements, relations, views, and documentation fields are written through MCP. No parallel diagram. No PlantUML as the deliverable. The model the organisation already keeps is the model the run writes. Inspect-before-create is part of the shared contract: specialists re-query identifiers, reuse existing elements where names match, and surface ambiguous near-matches instead of silent merges.
- 4) **The language stays in MCP resources.** Skills orchestrate tools and resources and do not embed the ArchiMate metamodel, which current models handle unreliably [10]. A structural check stops a skill citing a tool absent from the bridge. Allowed relationships, viewpoint definitions, and recipes stay on the MCP. If the language changes in Archi, skills do not carry a stale copy.
- 5) **Correspondence is a stage.** Traceability runs after layer specialists; quality assurance, layout, and documentation are terminal. Every created element carries an evidence line (stated, inferred, or existing) in its documentation field. Rationale lives in the model, not in a second artefact. The documentation specialist ends at a draft checkpoint; the first generation is not presented as finished work.

A prompt that says “draw ArchiMate” has none of these rules.

4.2 Pipeline

Fig. 1 shows the run: elicit intent, draft viewpoint selection, gate on the view plan; rejection returns to intent; approval dispatches only named layer specialists, then traceability, model QA, layout, and documentation.

Layer specialists: motivation, capability and strategy, business, application, technology and physical, implementa-

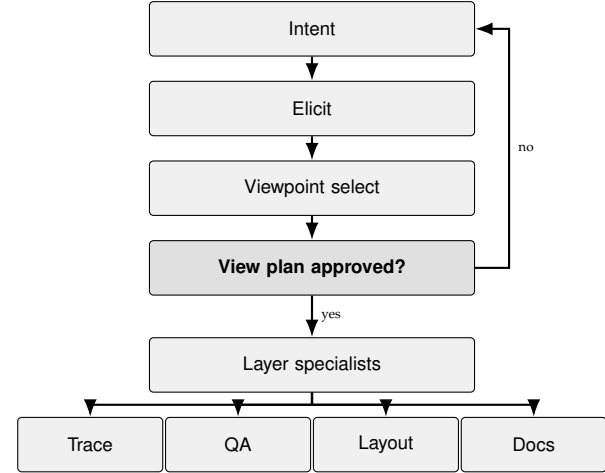


Fig. 1. SOAM pipeline. Nothing mutates until the view plan is approved. Trace, QA, layout, and documentation run after the named layer specialists.

tion and migration. The mill CRM brief includes motivation plus technology and physical, and omits implementation and migration. The range brief does the same and also refuses SysML and an air-vehicle product structure. Dispatch follows the plan, not a default of every layer.

4.3 Specialists

The user invokes only the orchestrator. The orchestrator dispatches the specialists in Table 2. A thirteenth skill in the published pack files pack defects; it is not part of SOAM and is out of this paper.

4.4 Shared modelling contract

The shared contract (CREATE_PATH) keeps twelve specialists from forking twelve create paths. Two gates are binding in prose here.

CP-G1, confirm-before-mutate: no mutating MCP call unless the view plan is approved. Without this gate the pipeline is a generator behind a chat UI.

CP-G7, first generation is a draft: the documentation specialist ends at a draft checkpoint. Completing a specialist does not finish the model.

Table 3 lists the contract gates. They exist because agent runs fail in ordinary ways: stale identifiers after context compaction, silent skips in a hand-off, invented elements with no source, notes placed before layout is closed, and a

TABLE 2
Orchestrator-dispatched specialists. Mutations only after view-plan approval.

Specialist	Mutates	Job
elicit	no	Normalise intent
viewpoint-select	no	Ground viewpoint to stakeholder and concern
motivation	yes	Drivers, goals, outcomes
capability / strategy	yes	Capabilities and resources
business	yes	Actors, processes, services
application	yes	Components and data objects
technology / physical	yes	Nodes, facilities, equipment
impl./migration	yes	Plateaus, gaps, work packages
traceability	yes	Cross-layer relations still open
model-qa	report first	Compliance; fixes only if approved
layout	yes	Archi-native layout on existing views
documentation	yes	Rationale fields; draft summary

TABLE 3
Shared modelling-contract gates (CREATE_PATH).

ID	Gate	Pass
CP-G1	Confirm-before-mutate	No mutate until the view plan is approved
CP-G2	Fresh IDs	Re-query IDs at start and after compaction
CP-G3	Disposition complete	Every candidate captured, folded, needs-user, or out-of-scope
CP-G4	No invention	Evidence line: stated, inferred, or existing
CP-G5	Annotate last	Notes and legends last on the view
CP-G6	Render close-out	Layout check, then inspect an exported view
CP-G7	First generation is a draft	Documentation ends at a draft checkpoint

specialist announcing the model finished because its own work ended.

These are method integrity rules, not a quality experiment. CP-G1 and CP-G7 are the ones that change organisational behaviour: nothing is written without approval, and the first generation is not finished work.

4.5 MCP as instrument

The JGS Archi Bridge runs inside Archi and exposes HTTP MCP at <http://127.0.0.1:18090/mcp> by default [5], [6], [22]. Offline inventory: 69 tools, 14 resources. Skills consume those only; they never modify the bridge or copy ArchiMate language tables into skill text. They read `archimate://reference/*` and `archimate://recipes/*` from the MCP. Archi 5.7 or later is the modelling surface [3], [4].

MCP makes the system of record agent-operable [6]. SOAM supplies the method on that surface. If the paper led with MCP, it would be a protocol note. Engineers should care about the gate and the canvas, not the port number.

4.6 Implementation

jgs-archi-skills v1.0.0 is MIT [4]. JGS Archi Bridge is the consume-only partner [5]. Helpers are Python 3.10 standard-library validators (viewpoint matrices, compliance asserts, structural checks), not a modelling engine. Skills are stateless. Model content and rationale live in Archi documentation fields. The repo holds skills, helpers, and tests. Live MCP is an opt-in smoke path, not evidence for this article.

The implementation is a method, not a prompt pack, for four choices already stated: the approval gate, specialist

Intent	Mill as-is plus CRM for order visibility
Stakeholders	Plant manager, sales, MES owner, CRM programme
In	Motivation, capabilities, operations, apps, tech, mill equipment
Out	Work packages, costing, MES rewrite
Views	Motivation Overview; Capability Map; Production Operations; Application Support; Technology and Physical
Specialists	Motivation, capability, business, application, technology-physical, then trace, QA, layout, docs
Constraints	OrderSight stays distinct from MillOS; equipment on a node; no roadmap
Gate	Approved or rejected (nothing written until yes)

Fig. 2. View-plan fields for the mill CRM brief. Approval opens MCP mutate. Rejection writes nothing.

dispatch as fragments, the consume-only language contract, and the waste-class mapping in Section 5.

From the architect's chair, the architect types plain-language intent into the orchestrator, not an element-type command. Elicit normalises stakeholders, concerns, scope in, and scope out. Viewpoint-select proposes views and names the specialists those views need. The orchestrator stops. The architect reads the view plan. Approval is the only thing that opens MCP mutate. Specialists then write into the open Archi model, reuse existing elements where names match, and leave traces, a QA report, a layout pass, and documentation fields. The documentation specialist stops at a draft. The architect still owns whether that draft is the architecture or a first cut to discard.

That is the cadence claim in operational form. The workshop, the unrecorded viewpoint, the week of boxes, and the late review are replaced by a gated pipeline that still cannot write without approval.

The view plan the architect is asked to sign is not a diagram. It is a short contract: intent restated, stakeholders and concerns, scope in and scope out, named views, named specialists, and the constraints the run must not violate (OrderSight stays distinct from MillOS; RangePlan stays distinct from GroundOS and AirStack; no SysML on this canvas). If that contract is wrong, the architect rejects it. Nothing has been written yet. That is cheaper than finding the wrong cut after fifty boxes exist.

Fig. 2 is the shape of that contract for the mill CRM brief. It sketches the fields; it is not a log of a live run.

4.7 What the three briefs dispatch

Table 4 puts the situational part of the method in one place. The mill CRM never calls implementation-migration. The range brief matches that cut and also refuses SysML and an air-vehicle product structure. The orchestrator does method composition, not template fill.

5 DEMONSTRATION

Evaluation follows Peffers: demonstration plus informed argument [24]. A reviewer should finish this section able to replay both walks and see each waste class hit by a

TABLE 4
Frozen briefs, views, specialists, and the constraint that would fail the run.

Brief	Views	Layer specialists	Hard constraint
Hatherley Plate	Motivation; Capability Map; Production Operations; Application Support; Technology and Physical	motivation, capability, business, application, technology-physical	OrderSight distinct from MilloS; equipment on a node; no roadmap
Moorfield Range	Motivation; Capability Map; Range Operations; Application Support; Technology and Physical	motivation, capability, business, application, technology-physical	RangePlan, GroundOS, AirStack distinct; no SysML; air vehicle not product structure

TABLE 5
Tool-as-canvas waste and the pipeline stage that addresses it.

Waste	What it costs	SOAM stage
Viewpoint chosen from habit or last week's view	Wrong audience, overloaded diagrams, rework	Viewpoint-select and view-plan approval
Elicitation and modelling decoupled	Notes go stale; review finds the wrong scope	Elicit, then plan; no mutate yet
Drawing labour as the main effort	Consistency and allowed relations stay in the modeller's head	Layer specialists via MCP
Cross-layer correspondence optional	Motivation to capability to application to technology to work package breaks first under deadline	Traceability
Documentation as a second artefact	Rationale in slides; model becomes a picture archive	Documentation fields in the model
Language knowledge not executable	Junior staff invent illegal relations; seniors lint	MCP resources and model-qa

named stage. Both walks were executed by the authors in Archi 5.7+ through the JGS Archi Bridge. Neither is a timed client engagement. Working models live in <https://github.com/jgsystemsconsulting/jgs-archi-skills-we>.

5.1 Waste classes and stages

Table 5 lists recurring cycle-time waste. Each row names the SOAM stage built for that waste.

SOAM runs elicit, plan, model, QA, and document as one governed run. Table 5 supports the cycle-time claim. No timing was done.

5.2 Walk A: Hatherley Plate

Hatherley Plate is a UK plate mill. Production runs on MilloS (MES) and WorksERP; sales still track promises on PromiseSheet. OrderSight is funded for order visibility. Plant engineers worry OrderSight will be drawn as if it replaces MilloS. Stakeholders: plant manager, head of sales, MES owner, CRM programme manager, operations planner, integration lead. Concerns: order visibility from promise to mill schedule; OrderSight must not absorb MilloS; equipment and the mill building stay in scope; what PlantGate connects. Scope in: motivation for visibility, capabilities, production operations, applications, technology nodes, physical equipment or facility. Scope out: work-package roadmap, product costing, greenfield MES rewrite. Current state: MilloS schedules the mill; WorksERP holds works orders; PromiseSheet holds customer promises; OrderSight

is in pilot; PlantGate sits in the plant comms room. Target: mill equipment and MilloS still visible; OrderSight sales-facing, served by PlantGate; shared order identity, not one merged blob.

Views: Motivation Overview, Capability Map, Production Operations, Application Support, Technology and Physical. Specialists: elicit, viewpoint-select, motivation, capability-strategy, business, application, technology-physical, traceability, model-qa, layout, documentation. Pass checks: listed views; OrderSight and MilloS remain separate applications; WorksERP, PromiseSheet, and PlantGate stay distinct; at least one facility or equipment element on a mill-floor node; confirmation gate before mutate.

This is the engineering-organisation case: a funded programme collides with mill operations. Sales wants order visibility. The mill already has a MES that schedules production and an ERP that holds works orders. If an architect or agent draws OrderSight as the system that runs the mill, the model would be used against the plant. The method must keep OrderSight and MilloS separate, keep mill equipment on a node, and still show PlantGate in the comms room connecting sales-facing identity to works orders. Pictures of the five views sit on [hatherley.html](#). Author-run frozen firm, not a client model.

Motivation is in for visibility; technology and physical are in for mill equipment and PlantGate; implementation and migration are out because the brief forbids a work-package roadmap. The hard constraints are method tests: if OrderSight absorbs MilloS on the canvas, the run failed even if the diagram has many elements. Correspondence here is equipment assigned to a node and refusal to merge two applications the plant does not treat as one. Modelling every layer would add a migration plateau the brief excluded. Dispatch keeps that plateau off the plan.

5.3 Walk B: Moorfield Range

Hawker Range Systems Ltd runs one UK training range at Moorfield. Instructors still track sorties on SortieBoard. RangePlan is funded for sortie visibility. The plant analogue: RangePlan must not be drawn as GroundOS (GCS software) or as AirStack (airborne mission computer). Stakeholders: range manager, chief instructor, GCS owner, planning programme manager, sortie planner, integration lead. Concerns: sortie visibility from tasking to live GCS; RangePlan must not absorb GroundOS or AirStack; hangar, GCS van, and range equipment stay in scope; what LinkGate connects.

Scope in: motivation for visibility, capabilities, range operations, applications, technology nodes, physical equipment or facility. Scope out: work-package roadmap, second site, GroundOS rewrite, weapons, ERP, air-vehicle SysML. Current state: GroundOS owns the live sortie; AirStack is the airborne mission computer; SortieBoard is the day's spreadsheet; RangePlan is in pilot; LinkGate sits in the comms cabin. Target: RangePlan, GroundOS, and AirStack stay separate; shared sortie identity, not one merged blob; LinkGate bridges tasking to GroundOS and does not talk to the air vehicle.

Views: Motivation Overview, Capability Map, Range Operations, Application Support, Technology and Physical. Specialists match the mill cut: motivation and technology-physical in; implementation and migration out. Pass checks: listed views; RangePlan, GroundOS, and AirStack remain separate applications; at least one facility or equipment element on a node; air vehicle not modelled as product structure; SysML stays off this canvas; confirmation gate before mutate.

Moorfield is the refuse-scope walk the paper still needs: dispatch does not invent SysML, a work-package plateau, or a merged blob. Pictures are not on hatherley.html; the working model is in the same we-repo named in the demonstration opener. Author-run frozen firm, not a client model. Without this control, SOAM acts as a generator that always fills every layer and every neighbouring notation.

5.4 Method integrity

A structural validator in the skill pack (`helpers/validate_skill_mcp_refs.py`) checks skill files cite only tools and resources in the bridge inventory. On the v1.0.0 pack it reports fourteen skill files and zero unknown references. That warrants the tool-cite claim, not modelling quality.

The copy-free language contract is a consume-only policy, not a validator result: skills consume MCP tools and resources only and never copy ArchiMate language tables into skills; they read `archimate://reference/*` and `archimate://recipes/*` from the MCP. Confirm-before-mutate remains CP-G1. These are integrity properties of the implementation, not an experiment on diagram quality.

6 LIMITS AND ADOPTION

SOAM is built so a shop that already runs Archi can install it: MIT skills, no second EA suite, architect on the gate. That is a design choice for industry uptake, not evidence that industry has taken the method up. Treating design-for-adoption as a finding would overstate the evidence.

Both walks are author-run Archi models, not client engagements. They show method coverage, not organisational uptake. Starter prompts for invoice-to-cash and a landing-zone refuse exist in the skill pack and are not this paper's evidence. The implementation is one bridge, one skill suite, Archi only; the argument does not generalise to Sparx or LeanIX. Language-model output is non-deterministic: the same intent can yield different element names. Governance (approval gate, QA, no-invention rule) is the control, not model temperature. Assistant users over-trust output when

TABLE 6
Threats to validity for this demonstration.

Threat	What it means
Lab scenarios	Both walks author-run, not client. Starter prompts for invoice-to-cash and landing-zone refuse are pack copy, not evidence. Coverage, not uptake.
Single implementation	One bridge, one skill suite, Archi only. Not Sparx, not LeanIX.
LLM non-determinism	Same intent can yield different element names. Governance is the control, not temperature.
Author as evaluator	Walks are by the method authors. Demonstration, not an independent trial.
Throughput unmeasured	Cycle-time claim is reasoned from Table 5. No timing was done.

verification is left to them [27]. Hallucination is a standing failure mode of language models [28]. The walks are by the method authors; this is demonstration, not an independent trial. Throughput is unmeasured. Adjacent measured evidence: a randomised GitHub Copilot trial completed coding tasks 55.8 percent faster [29]. That number is about coding, not modelling. The cycle-time claim is reasoned from waste removal in Table 5.

A later study should time hand modelling against SOAM on the same brief with a second architect, report one anonymised client run, measure whether repeated runs keep the same element names and view titles, and check whether QA reports the same defect classes a senior architect would. Those are future measurements, not results.

Table 6 restates those limits in one place.

Until that study exists, an engineering organisation can still decide whether to install the method. The test is local: open Archi, start the bridge, install the skills, run one frozen brief against a throwaway model, and ask whether the view plan is something an architect would sign. If the plan is not worth signing, the method failed before any box was placed. That is the point of the gate.

A trial worth doing in a week: pick one live change already in the backlog, not a greenfield scenario. Run SOAM against a copy of the current Archi model. Keep the original model closed. Compare the view plan to what the architect would have drawn by hand: named views, named exclusions, named constraints. Then look at correspondence. If the pipeline cannot keep OrderSight distinct from Millos on a plant brief, or merges RangePlan into GroundOS on a range brief, stop. Those are method failures, not taste. If the plan is signable and the constraints hold, the remaining question is labour: how long the gated run took versus the last time this team modelled a similar cut. That comparison is the follow-up paper, not this one.

7 TAKEAWAYS

- SOAM is a human-governed method that dispatches specialists after an approved view plan, not a prompt pack that draws diagrams.
- The system of record stays Archi; the ArchiMate language stays in MCP resources; the agent does not fork the metamodel.
- Instrument the method in the model you already keep. Put the architect on a view-plan gate before

any mutate. Treat missing traces, missing layout, and slide-deck rationale as pipeline failures, not cleanup.

- Cadence of architecture work is why an engineering organisation should care. Throughput is a relevance argument from removed waste classes, not a timed finding. Measure it later. Do not invent a percentage now.

REFERENCES

- [1] ISO/IEC/IEEE, *Software, systems and enterprise: Architecture description*, Geneva, Switzerland, 2022.
- [2] The Open Group, *ArchiMate 3.2 Specification*, Reading, U.K., 2022.
- [3] P. Beauvoir and J.-B. Sarrodie, "Archi: The open source modelling toolkit for creating archimate models," <https://www.archimatetool.com/>, 2026.
- [4] JG Systems Consulting Ltd, "jgs-archi-skills v1.0.0," <https://github.com/jgsystemsconsulting/jgs-archi-skills>, 2026.
- [5] —, "JGS Archi Bridge," <https://github.com/jgsystemsconsulting/jgs-archi-mcp>, 2026.
- [6] Anthropic and Model Context Protocol contributors, "Model context protocol specification (2025-06-18)," <https://modelcontextprotocol.io/specification/2025-06-18>, 2025.
- [7] J. Cámara, J. Troya, L. Burgueño, and A. Vallecillo, "On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML," *Softw. Syst. Model.*, vol. 22, pp. 781–793, 2023.
- [8] H.-G. Fill, P. Fettke, and J. Köpke, "Conceptual modeling and large language models: Impressions from first experiments with ChatGPT," *Enterp. Model. Inf. Syst. Archit.*, vol. 18, no. 3, 2023.
- [9] B. Nast, T. Arlt, J. Dakowski, and K. Sandkuhl, "Large language models for generating ArchiMate models," in *Proc. BIS 2026*, ser. Lecture Notes in Business Information Processing. Heidelberg, Germany: Springer, 2026, pp. 17–31.
- [10] F. Muff and H.-G. Fill, "Limitations of ChatGPT in conceptual modeling: Insights from experiments in metamodeling," in *Proc. Modellierung 2024, Workshop Modeling in the Age of LLMs*. Bonn, Germany: Gesellschaft für Informatik, 2024.
- [11] M. Lankhorst, *Enterprise Architecture at Work*, 4th ed. Heidelberg, Germany: Springer, 2017.
- [12] D. Greefhorst and E. Proper, *Architecture Principles: The Cornerstones of Enterprise Architecture*. Heidelberg, Germany: Springer, 2011.
- [13] H. A. Proper and M. M. Lankhorst, "Enterprise architecture: Towards essential sensemaking," *Enterp. Model. Inf. Syst. Archit.*, vol. 9, no. 1, pp. 5–21, 2014.
- [14] H. Jonkers, M. M. Lankhorst, H. W. L. ter Doest, F. Arbab, H. Bosma, and R. J. Wieringa, "Enterprise architecture: Management tool and blueprint for the organisation," *Inf. Syst. Front.*, vol. 8, no. 2, pp. 63–66, 2006.
- [15] The Open Group, *The TOGAF Standard, 10th Edition*, Reading, U.K., 2022.
- [16] D. L. Moody, "The 'physics' of notations: Toward a scientific basis for constructing visual notations in software engineering," *IEEE Trans. Softw. Eng.*, vol. 35, no. 6, pp. 756–779, 2009.
- [17] D. Quartel, W. Engelsman, H. Jonkers, and M. van Sinderen, "A goal-oriented requirements modelling language for enterprise architecture," in *Proc. EDOC 2009*. IEEE, 2009, pp. 3–13.
- [18] S. Hacks, H. Höfert, J. Salentin, Y. C. Yeong, and H. Lichter, "Towards the definition of enterprise architecture debts," in *Proc. EDOCW 2019*. IEEE, 2019, pp. 9–16.
- [19] J. Di Rocco, D. Di Ruscio, C. Di Sipio, P. T. Nguyen, and R. Rubei, "On the use of large language models in model-driven engineering," *Softw. Syst. Model.*, vol. 24, no. 3, pp. 923–948, 2025.
- [20] H.-G. Fill, J. Horkoff, P. Fettke, and J. Köpke, "Generative AI and conceptual modeling," *Bus. Inf. Syst. Eng.*, vol. 68, no. 1, pp. 1–5, 2026.
- [21] J. Chen and L. Zhao, "AI-driven innovation in enterprise architecture: A multi-agent system approach to adaptive design," in *Proc. EITCE 2024*. New York, NY: ACM, 2024, pp. 768–774.
- [22] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model context protocol (MCP): Landscape, security threats, and future research directions," [arXiv:2503.23278](https://arxiv.org/abs/2503.23278), 2025.
- [23] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Q.*, vol. 28, no. 1, pp. 75–105, 2004.
- [24] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *J. Manag. Inf. Syst.*, vol. 24, no. 3, pp. 45–77, 2007.
- [25] S. Brinkkemper, "Method engineering: engineering of information systems development methods and tools," *Inf. Softw. Technol.*, vol. 38, no. 4, pp. 275–280, 1996.
- [26] J. Ralyté, R. Deneckère, and C. Rolland, "Towards a generic model for situational method engineering," in *Proc. CAiSE 2003*, ser. LNCS, vol. 2681. Springer, 2003, pp. 95–110.
- [27] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" in *Proc. CCS 2023*. New York, NY: ACM, 2023, pp. 2785–2799.
- [28] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.
- [29] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirel, "The impact of AI on developer productivity: Evidence from GitHub Copilot," [arXiv:2302.06590](https://arxiv.org/abs/2302.06590), 2023.